Efficient transit network design and frequencies setting multi-objective optimization by Alternating Objective Genetic Algorithm

Renato Oliveira Arbex, Claudio Barbieri da Cunha

Problem statement / 1

Definition

The *Transit Network Design and Frequency Setting Problem* (**TNDFSP**)

- requires to compute a set of Service Lines (*routes* + *trip frequencies*)
- given the Infrastructure Network and its associated Demand Matrix.

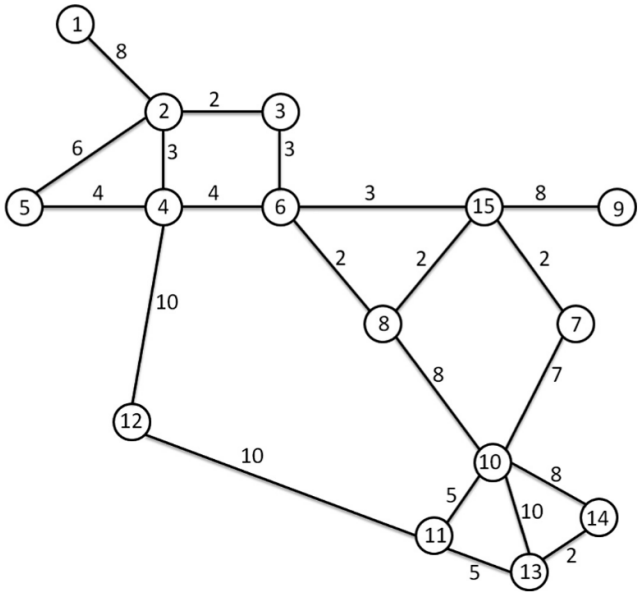


Figure: Example of INPUT Network

Origin/ Destination	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	0	400	200	60	80	150	75	75	30	160	30	25	35	0	0
2	400	0	50	120	20	180	90	90	15	130	20	10	10	5	0
3	200	50	0	40	60	180	90	90	15	45	20	10	10	5	0
4	60	120	40	0	50	100	50	50	15	240	40	25	10	5	0
5	80	20	60	50	0	50	25	25	10	120	20	15	5	0	0
6	150	180	180	100	50	0	100	100	30	880	60	15	15	10	0
7	75	90	90	50	25	100	0	50	15	440	35	10	10	5	0
8	75	90	90	50	25	100	50	0	15	440	35	10	10	5	0
9	30	15	15	15	10	30	15	15	0	140	20	5	0	0	0
10	160	130	45	240	120	880	440	440	140	0	600	250	500	200	0
11	30	20	20	40	20	60	35	35	20	600	0	75	95	15	0
12	25	10	10	25	15	15	10	10	5	250	75	0	70	0	0
13	35	10	10	10	5	15	10	10	0	500	95	70	0	45	0
14	0	5	5	5	0	10	5	5	0	200	15	0	45	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure: Example of INPUT Demand

Problem statement / 2

Definition

The *Transit Network Design and Frequency Setting Problem* (**TNDFSP**)

- requires to compute a set of Service Lines (*routes* + *trip frequencies*)
- given the Infrastructure Network and its associated Demand Matrix.

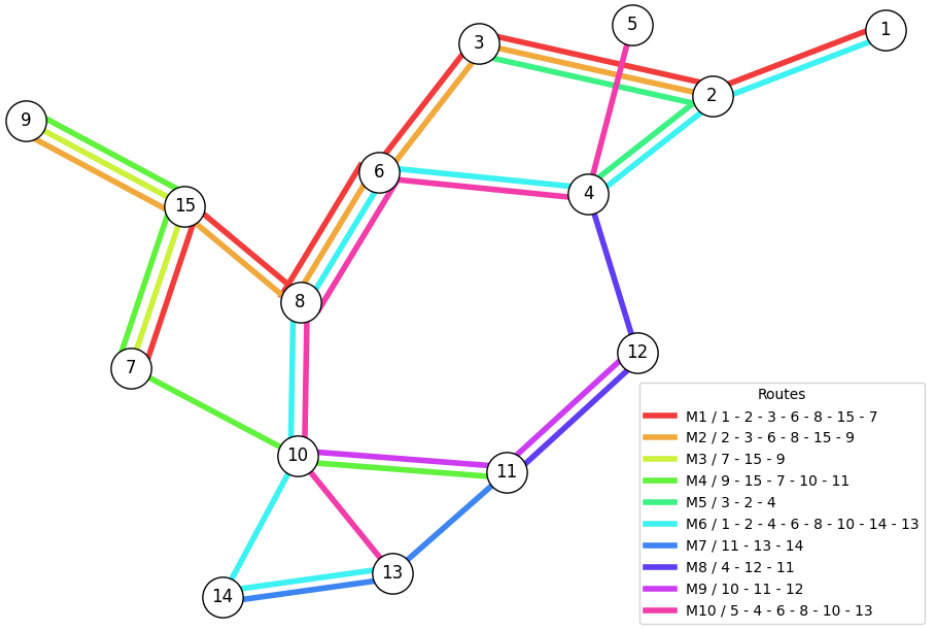


Figure: Example of OUTPUT Lines

Problem statement / 3

User Cost / Total for every Origin-Destination pair of:

What	Why
Average waiting time for vehicles	passengers waiting
Average in-vehicle travel time	passengers traveling
Average number of one-transfer trips	passengers changing vehicle 1 time
Average number of two-transfer trips	passengers changing vehicle 2 times
Unsatisfied demand	passengers overcrowded

Company Cost / Total for every Route of:

What	Why
Fleet size	maintenance and order

Problem statement / 4

Constraints

- A solution is a set of unique Service Lines forming a connected component covering the entire input network.
- A route is a path (of min length = 3) from a node A to a node B without repetitions of nodes.

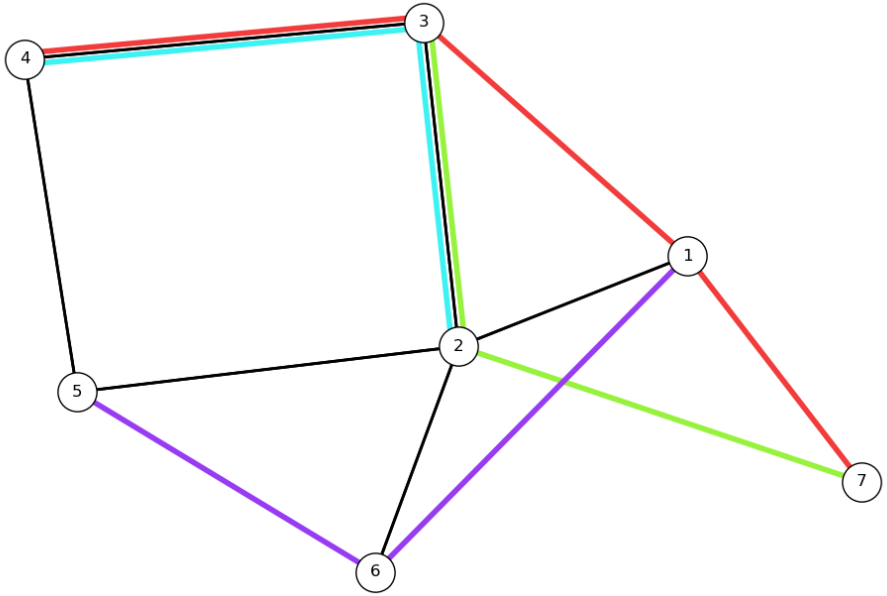


Figure: Valid Solution

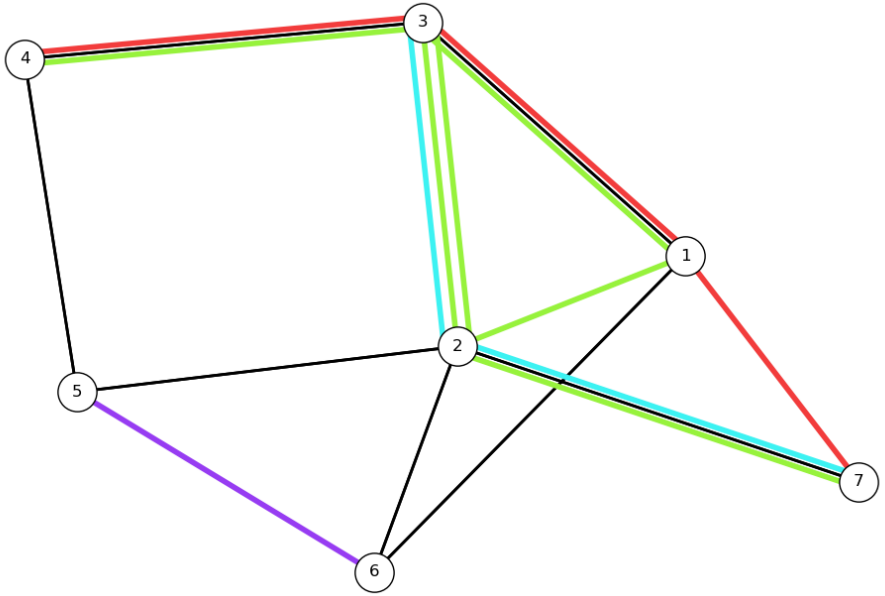
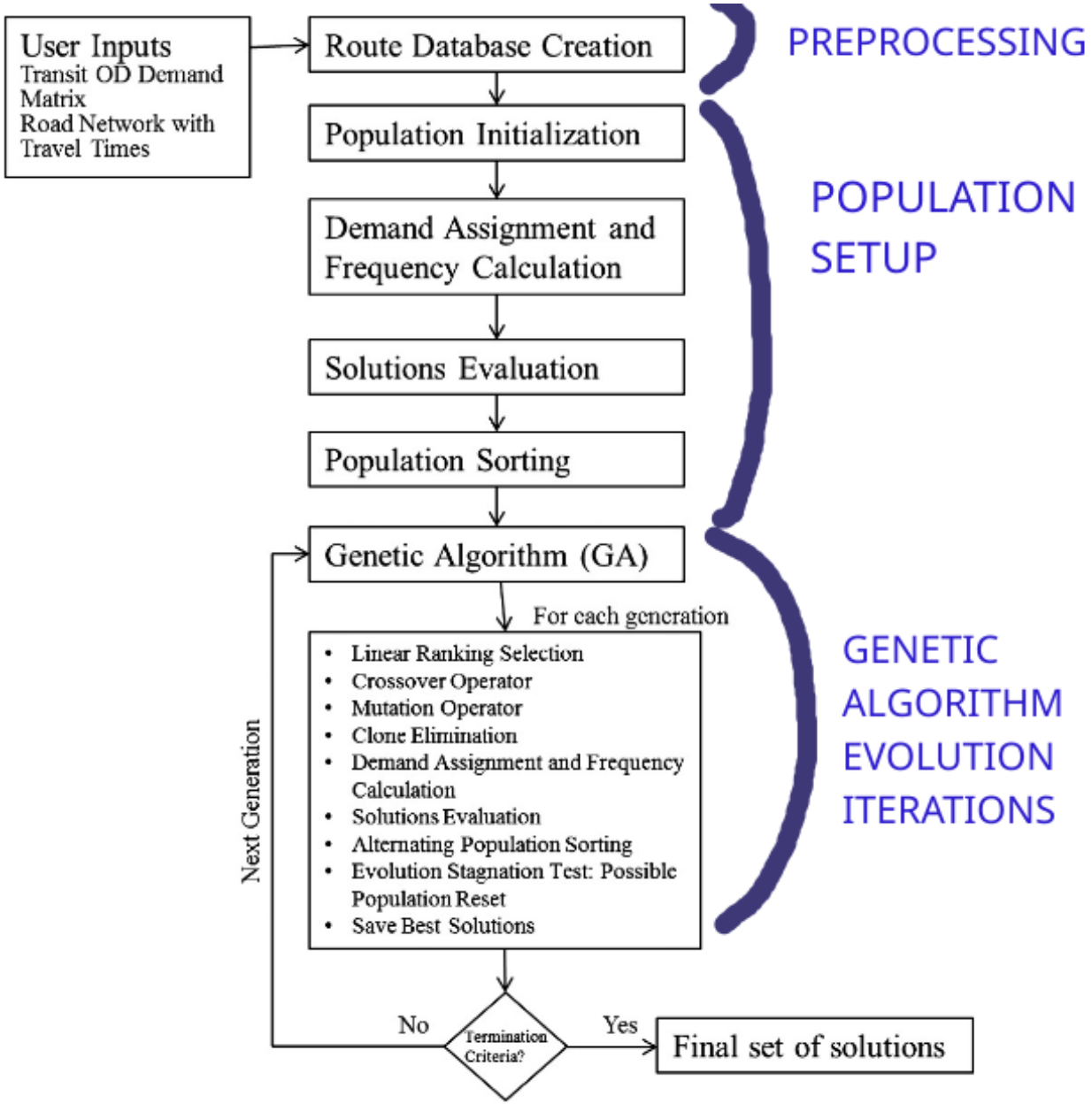


Figure: Invalid Solution

A Genetic Algorithm for TNDFSP



Route Database

Definition

- The Route DB is a collection of unique paths (each one with an ID).
- The Route DB is indexed so that it is possible to retrieve in $O(1)$ all the paths starting with node v, ending with node w or containing the edge (v, w)

Construction

- For each pair of nodes $(v, w) \in V \times V$ such that $OD[v, w] \neq 0$, add the *k-shortest-paths* from v to w (using a *detour ratio* for deriving k).
- Obtained using the **Yen** Algorithm, which in turn uses the **Dijkstra** algorithm on increasingly modified versions of the adjacency matrix of the graph.

Population Initialization / 1

Individual's Generation

- An *isolated node* has only 1 edge connecting it to the network (e.g. nodes 1, 9)
- For each *isolated node* v, add a random route starting/ending with v
- Track down which nodes are *served* by the current solution
- Until all nodes are served: randomly select a pair of unserved nodes v, w such that $OD[v, w] \leq 0$
- Add a random route containing the edge v, w
- Everytime check if the route is already in the solution (then select another one)
- If not all nodes can be served, the individual is discarded and a new one is generated instead

Population Initialization / 2

Individual's Feasibility Criteria

- Each node of the graph must be served and the individual shall form a connected component

Example of Individual

- R_1 : 1-2-3-6-8-10-11-13
- R_2 : 9-15-7-10-11-12
- R_3 : 7-15-8-6-3-2-4-5
- R_4 : 2-4-6-8-10-11-13-14
- R_5 : 13-14-10-8-6-3-2-4
- R_6 : 1-2-5-4-12
- R_7 : 11-10-7-15-6-3-2-1
- R_8 : 5-4-6-8-10-11
- R_9 : 13-11-12-4-5-2-1
- R_{10} : 9-15-8-6-3-2-4-12

Demand Assignment and Frequency Setting

- For each pair v, w such that $OD[v, w] \neq 0$, compute the probabilities P_j of a user wanting to travel from v to w using the combination of routes j.
- Those probabilities can be aggregated on route-level with demand from OD Matrix to compute the peak load of the route Q_r^{max}, and then the frequency f_r.
- NB: This becomes a subordinate optimization problem, which is solved by in/decrementing frequencies, recomputing peak load and re-updating frequencies, and so on, until convergence or with a iteration bound
- NB: Realistically, frequency shall relate to an integer number of trips during the hour/day period.

Solution Evaluation

Example of Frequency Setting and Individual Evaluation (see 9)

Route	Trip / Hour	Flee Size
R_1	10.51	12
R_2	8.42	9
R_3	6.78	4
R_4	9.4	9
R_5	9.0	8
R_6	3.72	3
R_7	13.0	13
R_8	11.9	9
R_9	4.27	6
R_{10}	4.34	4

Objective Function	Value
User Cost	458152
Company Cost	77

(Alternating) Population Sorting and Linear Ranking Selection

- The population is sorted by one of the two objective functions (F_1 and F_2), alternatively
- If in iteration t it was sorted with F_1, in iteration $t+1$ it is sorted with F_2

$$\ldots \Rightarrow \text{User Cost} \Rightarrow \text{Company Cost} \Rightarrow \text{User Cost} \Rightarrow \ldots$$

- The top k individuals out of the sorted population are selected for breeding, the rest is discarded

Crossover Operator

- A single point of crossover k is selected randomly
- Offsprings are created by exchanging their k-th route
- Satisfability of offspring is checked and in case of damage, a new crossover point k' is selected
- If no more offspring can be generated, a new pair of parents is selected ...
- ... until at most 80% of possible pairs has been processed

Mutation Operator

- An individual mutates by replacing one of its routes with another one from the Route DB
- The new route is selected so that is serves the same OD Pair that originally generated the route
- Again, if the individual cannot be mutated without having an invalid offspring, a new individual is selected for mutation

Clone Elimination, Stagnation Test and Population Reset

- If two individuals are (symmetrically) equal, one is discarded and replaced by a newly generated one
- (Test) Whenever the lowest fleet and user cost is not enhanced significantly after more than 200 generations ...
- (Reset) ... the entire population is renewed using the initial population procedure
- The best *k-th* individuals (regardless if they are offsprings or parents) are pushed to the next generation (*elitism*)

Balance of Terror / 1

Advantages

- This NP-Hard problem is reduced to a version of knapsack in which items are routes and quantities are frequencies
- Uses the Yen's Algorithm to obtain the building blocks of the Service Network and manipulates only their IDs
- Individuals can be made with a variable (or fixed or capped) number of Service Lines

Problems

- The Route DB requires a lot of space as it needs to hold around $O(k|V|^2)$ paths
- After the generation of the Route DB, no more new routes are created, because crossover and mutation are mere exchanges of routes
- No convergence guarantee as other Genetic Algorithms

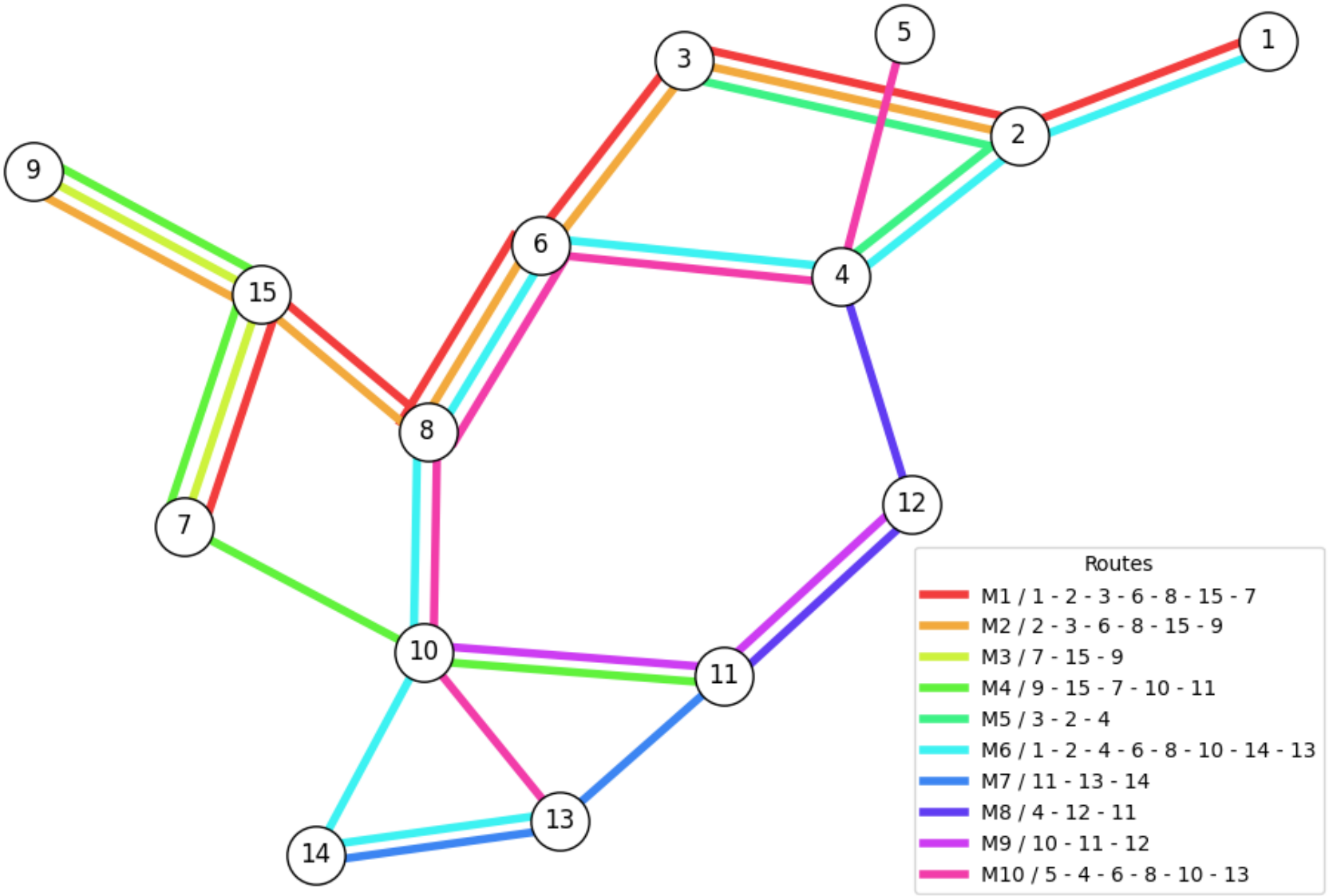
Balance of Terror / 2

Advantages

- The individuals generated by design are small in size, containing $O(|V|)$ "characters" (route IDs)
- The connected-component constraint further restricts the complexity of crossover/mutation operators
- Individuals can be pre-processed to compute indexes which allow to restrict the possible combinations of crossover/mutations

Problems

- Evaluation and Frequency Setting phases require to compute potentially every route between two points, which is massive
- All phases need to be carefully optimized (mostly with indexes and preprocessing) to prevent inefficient and redundant operations
- Some indexes to be effective require a lot of space (e.g. Route DB's "contains node" index)


1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
Routes
M1 / 1 - 2 - 3 - 6 - 8 - 15 - 7
M2 / 2 - 3 - 6 - 8 - 15 - 9
M3 / 7 - 15 - 9
M4 / 9 - 15 - 7 - 10 - 11
M5 / 3 - 2 - 4
M6 / 1 - 2 - 4 - 6 - 8 - 10 - 14 - 13
M7 / 11 - 13 - 14
M8 / 4 - 12 - 11
M9 / 10 - 11 - 12
M10 / 5 - 4 - 6 - 8 - 10 - 13

Thank You